

# Real-Time Airline Reservation System

## 1. Problem Statement

An airline requires a Real-Time Airline Reservation System to efficiently manage passenger information, flight schedules, aircraft details, seat availability, bookings, and payment information. Separate files or systems can cause data redundancy, inconsistency, difficult data retrieval, delayed seat availability updates, and potential double booking of seats. The proposed database system provides an organized relational solution with integrity constraints, SQL queries, views, indexes, transactions, and concurrency control.

## 2. Objective

- To design a relational database for airline reservation operations.
- To identify entities, relationships, keys, and integrity constraints.
- To store and retrieve passenger, flight, aircraft, seat, booking, and payment information.
- To query flight and seat availability and retrieve passenger bookings.
- To create a booking view and suitable indexes for efficient flight searching.
- To prevent double booking using database constraints and transaction/concurrency control.

## 3. Requirements and Environment Used

| Component              | Details                                                     |
|------------------------|-------------------------------------------------------------|
| DBMS / Platform        | Supabase with PostgreSQL                                    |
| SQL Interface          | Supabase SQL Editor                                         |
| Database Schema        | public                                                      |
| Main Entities          | Passenger, Aircraft, Flight, Seat, Booking, Payment         |
| Query Language         | SQL                                                         |
| Indexes                | B-tree indexes on Flight departure time and route/departure |
| Concurrency Protection | Unique partial index + transactions                         |
| Testing                | Supabase Table Editor and SQL query results                 |

## 4. Design / Proposed Solution

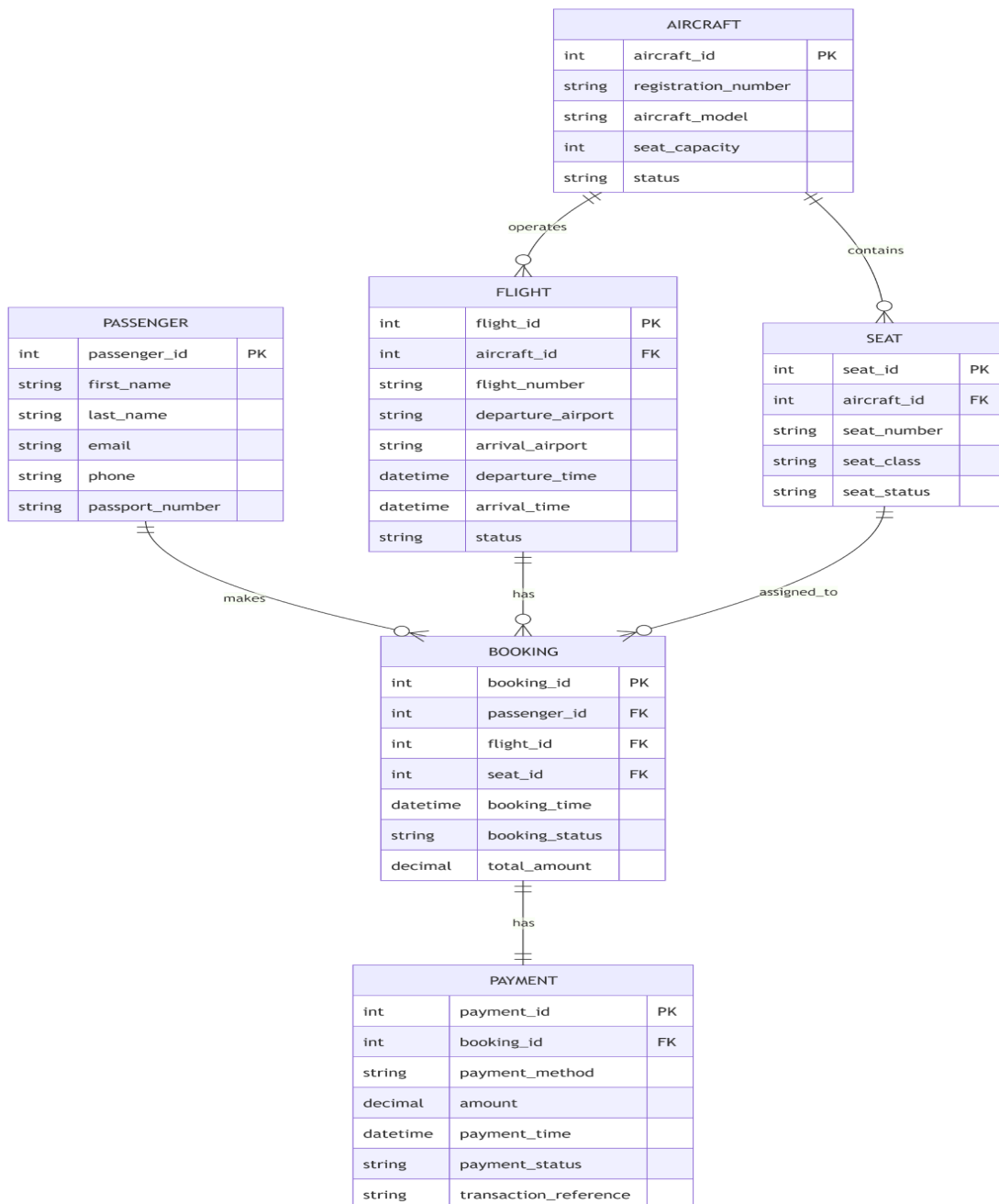
The database follows a relational design in which passenger, aircraft, flight, seat, booking, and payment data are stored in separate relations. Primary keys uniquely identify records, while foreign keys maintain relationships between tables. CHECK and UNIQUE constraints enforce valid values and data integrity.

### 4.1 Entities and Attributes

| Entity    | Primary Key  | Important Attributes               |
|-----------|--------------|------------------------------------|
| Passenger | Passenger_ID | Passenger_Name, Passport_No, Phone |
| Aircraft  | Aircraft ID  | Model, Capacity                    |

|         |            |                                                                               |
|---------|------------|-------------------------------------------------------------------------------|
| Flight  | Flight_ID  | Flight_Number, Source, Destination, Departure_Time, Arrival Time, Aircraft ID |
| Seat    | Seat_ID    | Aircraft_ID, Seat_Number, Class, Status                                       |
| Booking | Booking_ID | Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Booking Status                |
| Payment | Payment_ID | Booking_ID, Amount, Payment Status                                            |

## 4.2 EER Diagram



### 4.3 Relationships and Cardinalities

| Relationship        | Cardinality | Meaning                                                                      |
|---------------------|-------------|------------------------------------------------------------------------------|
| Aircraft – Flight   | 1 : M       | One aircraft can operate many flights.                                       |
| Aircraft – Seat     | 1 : M       | One aircraft contains many seats.                                            |
| Passenger – Booking | 1 : M       | One passenger can have many bookings.                                        |
| Flight – Booking    | 1 : M       | One flight can have many bookings.                                           |
| Seat – Booking      | 1 : M*      | A seat can appear in bookings; active duplicate use is prevented per flight. |
| Booking – Payment   | 1 : 1       | Each booking has at most one payment record in this design.                  |

### 4.4 Relational Schema

PASSENGER(Passenger\_ID PK, Passenger\_Name, Passport\_No UNIQUE, Phone)

AIRCRAFT(Aircraft\_ID PK, Model, Capacity)

FLIGHT(Flight\_ID PK, Flight\_Number, Source, Destination,  
Departure\_Time, Arrival\_Time, Aircraft\_ID FK)

SEAT(Seat\_ID PK, Aircraft\_ID FK, Seat\_Number, Class, Status)

BOOKING(Booking\_ID PK, Passenger\_ID FK, Flight\_ID FK, Seat\_ID FK,  
Booking\_Date, Booking\_Status)

PAYMENT(Payment\_ID PK, Booking\_ID FK UNIQUE, Amount, Payment\_Status)

### 4.5 Keys and Integrity Constraints

- Primary keys uniquely identify each record.
- Foreign keys connect Flight to Aircraft, Seat to Aircraft, Booking to Passenger/Flight/Seat, and Payment to Booking.
- Passport\_No is UNIQUE.
- Aircraft Capacity must be greater than zero.
- Departure\_Time must be earlier than Arrival\_Time.
- Seat\_Number is unique for each aircraft.
- Class is restricted to ECONOMY, BUSINESS, or FIRST.
- Booking\_Status is restricted to PENDING, CONFIRMED, or CANCELLED.
- Payment\_Status is restricted to PENDING, PAID, or FAILED.
- A unique partial index on (Flight\_ID, Seat\_ID) for active bookings prevents double booking.

## **5. Algorithm / Pseudocode**

### **5.1 Flight Availability**

```
BEGIN
  Read source, destination, and required date/time
  Search FLIGHT for matching source and destination
  Select flights whose departure time is in the future
  Sort results by departure time
  Display available flights
END
```

### **5.2 Seat Availability**

```
BEGIN
  Read Flight_ID
  Find seats belonging to the aircraft assigned to the flight
  Select seats whose status is AVAILABLE
  Exclude seats already present in active bookings
  Display available seat number and class
END
```

### **5.3 Booking and Double-Booking Prevention**

```
BEGIN TRANSACTION
  Attempt to insert passenger, flight, and seat booking
  Database checks foreign keys and booking constraints
  Unique active flight-seat index checks whether the seat is already booked
  IF constraint succeeds
    COMMIT transaction
  ELSE
    ROLLBACK / reject booking
END
```

## **6. Implementation / Source Code**

The following SQL was implemented in the Supabase SQL Editor.

### **6.1 Table Creation**

```
CREATE TABLE passenger (
  passenger_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  passenger_name VARCHAR(100) NOT NULL,
  passport_no VARCHAR(30) NOT NULL UNIQUE,
  phone VARCHAR(15) NOT NULL
);

CREATE TABLE aircraft (
  aircraft_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  model VARCHAR(50) NOT NULL,
```

```
    capacity INT NOT NULL CHECK (capacity > 0)
);
```

```
CREATE TABLE flight (
    flight_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    flight_number VARCHAR(20) NOT NULL,
    source VARCHAR(50) NOT NULL,
    destination VARCHAR(50) NOT NULL,
    departure_time TIMESTAMP NOT NULL,
    arrival_time TIMESTAMP NOT NULL,
    aircraft_id BIGINT NOT NULL REFERENCES aircraft(aircraft_id),
    CHECK (departure_time < arrival_time)
);
```

```
CREATE TABLE seat (
    seat_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    aircraft_id BIGINT NOT NULL REFERENCES aircraft(aircraft_id),
    seat_number VARCHAR(10) NOT NULL,
    class VARCHAR(20) NOT NULL,
    status VARCHAR(20) DEFAULT 'AVAILABLE',
    UNIQUE (aircraft_id, seat_number),
    CHECK (class IN ('ECONOMY', 'BUSINESS', 'FIRST')),
    CHECK (status IN ('AVAILABLE', 'BLOCKED'))
);
```

```
CREATE TABLE booking (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    passenger_id BIGINT NOT NULL REFERENCES passenger(passenger_id),
    flight_id BIGINT NOT NULL REFERENCES flight(flight_id),
    seat_id BIGINT NOT NULL REFERENCES seat(seat_id),
    booking_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    booking_status VARCHAR(20) DEFAULT 'PENDING',
    CHECK (booking_status IN ('PENDING', 'CONFIRMED', 'CANCELLED'))
);
```

```
CREATE TABLE payment (
    payment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    booking_id BIGINT NOT NULL UNIQUE REFERENCES booking(booking_id),
    amount NUMERIC(10,2) NOT NULL CHECK (amount >= 0),
    payment_status VARCHAR(20) NOT NULL,
    CHECK (payment_status IN ('PENDING', 'PAID', 'FAILED'))
);
```

## 6.2 Booking View

```
CREATE OR REPLACE VIEW passenger_booking_view AS
SELECT
    b.booking_id,
    p.passenger_id,
    p.passenger_name,
    f.flight_number,
    f.source,
    f.destination,
    f.departure_time,
    f.arrival_time,
    s.seat_number,
    s.class,
    b.booking_date,
    b.booking_status,
    pay.amount,
    pay.payment_status
FROM booking b
JOIN passenger p ON b.passenger_id = p.passenger_id
JOIN flight f ON b.flight_id = f.flight_id
JOIN seat s ON b.seat_id = s.seat_id
LEFT JOIN payment pay ON b.booking_id = pay.booking_id;
```

## 6.3 Index Design

```
CREATE INDEX idx_flight_departure_time
ON flight(departure_time);

CREATE INDEX idx_flight_route_departure
ON flight(source, destination, departure_time);

CREATE UNIQUE INDEX unique_active_flight_seat
ON booking(flight_id, seat_id)
WHERE booking_status IN ('PENDING', 'CONFIRMED');
```

## 6.4 Important SQL Queries

```
-- Flight availability
SELECT flight_id, flight_number, source, destination,
       departure_time, arrival_time
FROM flight
WHERE source = 'Chennai'
      AND destination = 'Delhi'
      AND departure_time > CURRENT_TIMESTAMP
ORDER BY departure_time;
```

```
-- Available seats for Flight 1
SELECT s.seat_id, s.seat_number, s.class
FROM seat s
JOIN flight f ON s.aircraft_id = f.aircraft_id
WHERE f.flight_id = 1
AND s.status = 'AVAILABLE'
AND NOT EXISTS (
    SELECT 1
    FROM booking b
    WHERE b.flight_id = 1
    AND b.seat_id = s.seat_id
    AND b.booking_status IN ('PENDING', 'CONFIRMED')
);
```

```
-- Passenger bookings
SELECT p.passenger_name, f.flight_number,
       f.source, f.destination, f.departure_time,
       s.seat_number, s.class, b.booking_status
FROM passenger p
JOIN booking b ON p.passenger_id = b.passenger_id
JOIN flight f ON b.flight_id = f.flight_id
JOIN seat s ON b.seat_id = s.seat_id
WHERE p.passenger_id = 1;
```

```
-- View output
SELECT * FROM passenger_booking_view;
```

## 7. Test Cases and Expected / Actual Results

| TC ID | Test                                    | Expected Result                                          | Actual Result                    | Status |
|-------|-----------------------------------------|----------------------------------------------------------|----------------------------------|--------|
| TC01  | Flight availability:<br>Chennai → Delhi | AI101 displayed                                          | AI101 displayed                  | PASS   |
| TC02  | Seat availability<br>for Flight 1       | Unbooked seats<br>displayed                              | 1B, 2B, 3A<br>displayed          | PASS   |
| TC03  | Passenger<br>booking view               | Combined<br>passenger/flight/s<br>eat/payment<br>details | 4 rows displayed                 | PASS   |
| TC04  | Flight indexes                          | Primary and<br>created indexes<br>listed                 | 3 flight indexes<br>listed       | PASS   |
| TC05  | Attempt duplicate<br>Flight 1 + Seat 1  | Booking rejected                                         | Unique constraint<br>error shown | PASS   |
| TC06  | Successful<br>transaction               | New booking<br>committed                                 | 5 bookings<br>displayed          | PASS   |

8. Execution Screenshots / Output

The following screenshots were captured from the Supabase SQL Editor/Table Editor during implementation.

The screenshot shows the Supabase Table Editor interface for the 'passenger' table in the 'public' schema. The table has four columns: 'passenger\_id' (integer), 'passenger\_name' (varchar), 'passport\_no' (varchar), and 'phone' (varchar). There are four records listed. The interface includes a sidebar with a list of tables (aircraft, booking, flight, passenger, passenger\_booking, payment, seat) and a top navigation bar with the project name 'jaiwanthy-v's Org' and a 'Connect' button.

| passenger_id | passenger_name | passport_no | phone      |
|--------------|----------------|-------------|------------|
| 1            | Rahul Kumar    | P123456     | 9876543210 |
| 2            | Priya Sharma   | P234567     | 9876543211 |
| 3            | Arun Kumar     | P345678     | 9876543212 |
| 4            | Sneha Patel    | P456789     | 9876543213 |

Figure 1. Passenger view output

The screenshot shows the Supabase Table Editor interface for the 'flight' table in the 'public' schema. The table has five columns: 'flight\_id' (integer), 'flight\_number' (varchar), 'source' (varchar), 'destination' (varchar), and 'departure\_time' (timestamp). There are three records listed. The interface is similar to the previous screenshot, showing the same sidebar and navigation bar.

| flight_id | flight_number | source  | destination | departure_time  |
|-----------|---------------|---------|-------------|-----------------|
| 1         | AI101         | Chennai | Delhi       | 2026-09-10 10:0 |
| 2         | AI202         | Mumbai  | Chennai     | 2026-09-10 15:0 |
| 3         | AI303         | Delhi   | Mumbai      | 2026-09-11 09:0 |

Figure 2. Flight view output



Table Editor

schema public

+ New table

Search tables...

aircraft UNRESTRICTED

booking UNRESTRICTED

flight UNRESTRICTED

passenger UNRESTRICTED

passenger\_bookin... UNRESTRICTED

payment UNRESTRICTED

seat UNRESTRICTED

Filter by aircraft\_id, model, capacity or ask AI

Sort

RLS disabled

Role postgres

Insert

|                          | aircraft_id i. | model varchar | capacity int4 |  |
|--------------------------|----------------|---------------|---------------|--|
| <input type="checkbox"/> | 1              | Airbus A320   | 180           |  |
| <input type="checkbox"/> | 2              | Boeing 737    | 190           |  |

Page 1 of 1

100 rows

2 records

Activate Windows

Go to Settings to activate Windows.

Data Definition

Figure 3. Aircraft view output

Table Editor

schema public

+ New table

Search tables...

aircraft UNRESTRICTED

booking UNRESTRICTED

flight UNRESTRICTED

passenger UNRESTRICTED

passenger\_bookin... UNRESTRICTED

payment UNRESTRICTED

seat UNRESTRICTED

Filter by seat\_id, aircraft\_id, seat\_number... or ask AI

Sort

RLS disabled

Role postgres

Insert

|                          | seat_id i. | aircraft_id i. | seat_number varchar | class varchar | status varchar |
|--------------------------|------------|----------------|---------------------|---------------|----------------|
| <input type="checkbox"/> | 1          | 1              | 1A                  | BUSINESS      | AVAILABLE      |
| <input type="checkbox"/> | 2          | 1              | 1B                  | BUSINESS      | AVAILABLE      |
| <input type="checkbox"/> | 3          | 1              | 2A                  | ECONOMY       | AVAILABLE      |
| <input type="checkbox"/> | 4          | 1              | 2B                  | ECONOMY       | AVAILABLE      |
| <input type="checkbox"/> | 5          | 1              | 3A                  | ECONOMY       | AVAILABLE      |
| <input type="checkbox"/> | 6          | 2              | 1A                  | BUSINESS      | AVAILABLE      |
| <input type="checkbox"/> | 7          | 2              | 1B                  | BUSINESS      | AVAILABLE      |
| <input type="checkbox"/> | 8          | 2              | 2A                  | ECONOMY       | AVAILABLE      |
| <input type="checkbox"/> | 9          | 2              | 2B                  | ECONOMY       | AVAILABLE      |

Page 1 of 1

100 rows

9 records

Activate Windows

Go to Settings to activate Windows.

Data Definition

Figure 4. Seat view output

jaiwanthy-v's Org FREE / airline-reservation-system / main PRODUCTION Connect

Feedback Search... Ctrl K

Table Editor

public.payment public.flight public.booking

schema public

+ New table

Search tables...

aircraft UNRESTRICTED

booking UNRESTRICTED

flight UNRESTRICTED

passenger UNRESTRICTED

passenger\_bookin... UNRESTRICTED

payment UNRESTRICTED

seat UNRESTRICTED

Filter by booking\_id, passenger\_id, flight\_id... or ask AI

Sort RLS disabled Role postgres Insert

|  | booking_id | passenger_id | flight_id | seat_id | booking_date               | booking_status |
|--|------------|--------------|-----------|---------|----------------------------|----------------|
|  | 1          | 1            | 1         | 1       | 2026-09-01 04:29:51.635456 | CONFIRMED      |
|  | 2          | 2            | 1         | 3       | 2026-09-01 04:29:51.635456 | CONFIRMED      |
|  | 3          | 3            | 2         | 6       | 2026-09-01 04:29:51.635456 | CONFIRMED      |
|  | 4          | 4            | 3         | 4       | 2026-09-01 04:29:51.635456 | PENDING        |
|  | 6          | 3            | 1         | 2       | 2026-09-01 04:56:19.495382 | CONFIRMED      |

Page 1 of 1 100 rows 5 records

Activate Windows Go to Settings to activate Windows.

Data Definition

Figure 5. Booking view output

jaiwanthy-v's Org FREE / airline-reservation-system / main PRODUCTION Connect

Feedback Search... Ctrl K

Table Editor

public.payment public.flight public.booking public.passenger

schema public

+ New table

Search tables...

aircraft UNRESTRICTED

booking UNRESTRICTED

flight UNRESTRICTED

passenger UNRESTRICTED

passenger\_bookin... UNRESTRICTED

payment UNRESTRICTED

seat UNRESTRICTED

Filter by payment\_id, booking\_id, amount... or ask AI

Sort RLS disabled Role postgres Insert

|  | payment_id | booking_id | amount  | payment_status |
|--|------------|------------|---------|----------------|
|  | 5          | 1          | 5500.00 | PAID           |
|  | 6          | 2          | 5500.00 | PAID           |
|  | 7          | 3          | 4200.00 | PAID           |
|  | 8          | 4          | 6000.00 | PENDING        |

Page 1 of 1 100 rows 4 records

Activate Windows Go to Settings to activate Windows.

Data Definition

Figure 6. Payment view output

The screenshot shows a database client interface with a SQL editor and a results pane. The SQL editor contains a query to create or replace a view named 'passenger\_booking\_view' and select data from it. The results pane shows 4 rows of data.

```

1 CREATE OR REPLACE VIEW passenger_booking_view AS
2 SELECT
3     b.booking_id,
4     p.passenger_id,
5     p.passenger_name,
6     f.flight_number,
7     f.source,
8     f.destination,
9     f.departure_time,
10    f.arrival_time,

```

| booking_id | passenger_id | passenger_name | flight_number | source  | destination | departure_time      | arrival_time        | seat_number | class    | booking_date            |
|------------|--------------|----------------|---------------|---------|-------------|---------------------|---------------------|-------------|----------|-------------------------|
| 1          | 1            | Rahul Kumar    | AI101         | Chennai | Delhi       | 2026-09-10 10:00:00 | 2026-09-10 13:00:00 | 1A          | BUSINESS | 2026-09-01 04:29:51.635 |
| 2          | 2            | Priya Sharma   | AI101         | Chennai | Delhi       | 2026-09-10 10:00:00 | 2026-09-10 13:00:00 | 2A          | ECONOMY  | 2026-09-01 04:29:51.635 |
| 3          | 3            | Arun Kumar     | AI202         | Mumbai  | Chennai     | 2026-09-10 15:00:00 | 2026-09-10 17:00:00 | 1A          | BUSINESS | 2026-09-01 04:29:51.635 |
| 4          | 4            | Sneha Patel    | AI303         | Delhi   | Mumbai      | 2026-09-11 09:00:00 | 2026-09-11 11:30:00 | 2B          | ECONOMY  | 2026-09-01 04:29:51.635 |

Figure 7. Passenger Booking view output

The screenshot shows a database client interface with a SQL editor and a results pane. The SQL editor contains a query to select index names and definitions from the 'pg\_indexes' table for the 'flight' table. The results pane shows 3 rows of data.

```

1 SELECT
2     indexname,
3     indexdef
4 FROM pg_indexes
5 WHERE tablename = 'flight';

```

| indexname                  | indexdef                                                                     |
|----------------------------|------------------------------------------------------------------------------|
| flight_pkey                | CREATE UNIQUE INDEX flight_pkey ON public.flight USING btree (flight_id)     |
| idx_flight_departure_time  | CREATE INDEX idx_flight_departure_time ON public.flight USING btree (depart  |
| idx_flight_route_departure | CREATE INDEX idx_flight_route_departure ON public.flight USING btree (source |

Figure 8. Flight indexes output

The screenshot shows a database query editor interface. The top bar includes the user 'jaiwanthy-v's Org', a 'FREE' label, the project name 'airline-reservation-system', and a 'main' database with a 'PRODUCTION' schema. A 'Connect' button is visible. The query editor contains the following SQL code:

```

1 SELECT
2     flight_id,
3     flight_number,
4     source,
5     destination,
6     departure_time,
7     arrival_time
8 FROM flight
9 WHERE source = 'Chennai'
10    AND destination = 'Delhi'
11    AND departure_time > CURRENT_TIMESTAMP
12 ORDER BY departure_time;

```

The results section shows 1 row (Limited to only 1000 rows). The output table is as follows:

| flight_id | flight_number | source  | destination | departure_time      | arrival_time        |
|-----------|---------------|---------|-------------|---------------------|---------------------|
| 1         | AI101         | Chennai | Delhi       | 2026-09-10 10:00:00 | 2026-09-10 13:00:00 |

Figure 9. Flight availability query output

The screenshot shows a database query editor interface. The top bar includes the user 'jaiwanthy-v's Org', a 'FREE' label, the project name 'airline-reservation-system', and a 'main' database with a 'PRODUCTION' schema. A 'Connect' button is visible. The query editor contains the following SQL code:

```

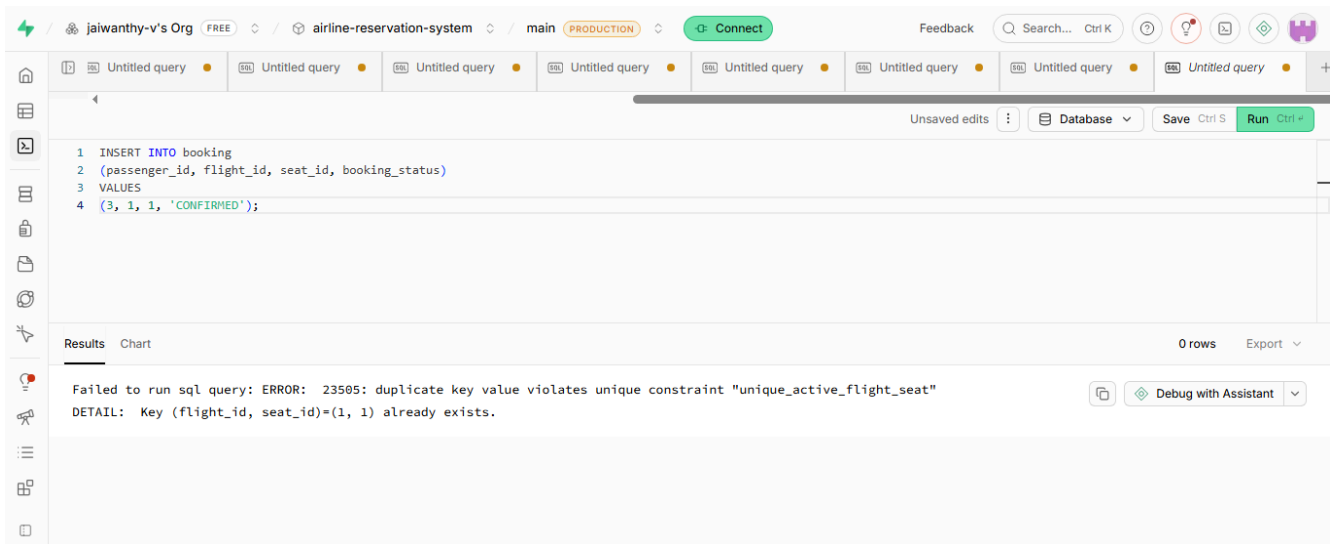
1 SELECT
2     s.seat_id,
3     s.seat_number,
4     s.class
5 FROM seat s
6 JOIN flight f
7     ON s.aircraft_id = f.aircraft_id
8 WHERE f.flight_id = 1
9    AND s.status = 'AVAILABLE'
10 AND NOT EXISTS (
11     SELECT 1
12     FROM booking b

```

The results section shows 3 rows (Limited to only 1000 rows). The output table is as follows:

| seat_id | seat_number | class    |
|---------|-------------|----------|
| 2       | 1B          | BUSINESS |
| 4       | 2B          | ECONOMY  |
| 5       | 3A          | ECONOMY  |

Figure 10. Available seats query output



*Figure 11. Double-booking prevention error*

```

1 SELECT *
2 FROM booking
3 ORDER BY booking_id;

```

Results Chart 5 rows (Limited to only 1000 rows) Export

| booking_id | passenger_id | flight_id | seat_id | booking_date               | booking_status |
|------------|--------------|-----------|---------|----------------------------|----------------|
| 1          | 1            | 1         | 1       | 2026-09-01 04:29:51.635459 | CONFIRMED      |
| 2          | 2            | 1         | 3       | 2026-09-01 04:29:51.635459 | CONFIRMED      |
| 3          | 3            | 2         | 6       | 2026-09-01 04:29:51.635459 | CONFIRMED      |
| 4          | 4            | 3         | 4       | 2026-09-01 04:29:51.635459 | PENDING        |
| 6          | 3            | 1         | 2       | 2026-09-01 04:56:19.495382 | CONFIRMED      |

*Figure 12. Booking table after successful transaction*

## 9. Analysis and Discussion

The implemented database successfully separates airline data into normalized relations, reducing duplication and improving consistency. Foreign-key constraints maintain valid relationships between entities. SQL joins provide combined reservation information, while the booking view simplifies repeated reporting. Indexes on departure time and route/departure support efficient flight searches. The unique partial index on active bookings prevents two active bookings from using the same seat on the same flight. The test of a duplicate booking was rejected by PostgreSQL, demonstrating effective integrity protection.

## **9.1 Transaction and Concurrency Control**

A reservation should be treated as an atomic transaction. The database starts a transaction, performs the booking operation, validates constraints, and commits only when the operation succeeds. If a constraint or other error occurs, the transaction can be rolled back. In this implementation, the unique partial index on (flight\_id, seat\_id) for PENDING and CONFIRMED bookings provides database-level protection against concurrent attempts to reserve the same seat for the same flight. PostgreSQL handles concurrent writes so that conflicting inserts cannot both satisfy the unique constraint.

## **10. Conclusion**

The Real-Time Airline Reservation System was designed and implemented as a relational database using Supabase PostgreSQL. The solution includes six related entities, primary and foreign keys, integrity constraints, SQL queries for flight and seat availability, passenger booking retrieval, a booking view, indexes, transactions, and double-booking prevention. The successful test results demonstrate that the database can support consistent and efficient airline reservation operations.

## **11. References**

- Supabase Documentation – PostgreSQL database and SQL Editor.
- PostgreSQL Documentation – CREATE TABLE, constraints, indexes, views, and transactions.
- Database Management Systems course notes and prescribed laboratory materials.

## **One-Page Write-up**

The Real-Time Airline Reservation System is a relational database solution designed to manage the core operations of an airline reservation process. The system maintains six major entities: Passenger, Aircraft, Flight, Seat, Booking, and Payment. Passenger stores traveller details, Aircraft stores aircraft model and capacity, Flight stores route and schedule information, Seat stores aircraft-specific seat details, Booking connects a passenger with a flight and seat, and Payment stores payment information associated with a booking.

The database was implemented using Supabase with PostgreSQL. Primary keys uniquely identify records and foreign keys maintain relationships between the tables. Additional constraints such as UNIQUE and CHECK constraints improve data integrity. SQL queries were developed to search flights, identify available seats, and retrieve passenger bookings. A booking view was created to combine passenger, flight, seat, booking, and payment details into a single reporting structure.

Indexes were created on departure time and on source, destination, and departure time to support faster flight searches. A major requirement of an airline reservation system is preventing two passengers from reserving the same seat on the same flight. This was addressed using a unique partial index for active bookings together with transactions. When a duplicate booking was attempted, PostgreSQL rejected it with a unique constraint violation, while a valid booking was successfully committed.

The implementation and test results show that the proposed database provides organized data storage, reliable relationships, efficient retrieval, and protection against double booking. The system can be extended in the future with user authentication, cancellation/refund processing, fare management, boarding passes, and an application interface.